

3blue1brown – LLM's

1. What GPT Means

GPT stands for **Generative Pretrained Transformer**.

- **Generative:**
The model isn't just classifying inputs, it **creates** new outputs – generating text, speech, or even images. For text, this means predicting and extending passages word by word.
 - **Pretrained:**
The model is first exposed to enormous datasets (billions of words, code, etc.), learning general language patterns. "Pretrained" suggests that this isn't the end – the model can be **fine-tuned** on smaller, specialized datasets for domain-specific tasks.
 - **Transformer:**
The word that matters most. A **transformer** is a specific kind of neural network introduced in 2017 by Google, originally for **machine translation**. It has since become the backbone of almost all modern AI models: ChatGPT, DALL·E, Whisper, MidJourney, Claude, and more.
-

2. From Prediction to Generation

The transformer's base task: **predict the next token** (a word or piece of a word).

- Output = probability distribution over possible next tokens.
 - But how does prediction become **generation**?
1. Provide **seed text**.
 2. Model predicts probability distribution for next token.
 3. Sample one token from this distribution.
 4. Append token to text.
 5. Repeat – predict again with the updated text.

This looping process transforms a predictive model into a **generative model**.

Example: GPT-2 vs GPT-3

- With GPT-2 (smaller model, running locally), generated stories often degenerate into **nonsense**.
- With GPT-3 (same structure but vastly larger: 175B parameters), stories suddenly become **coherent** and context-sensitive.

Illustration:

Prompt: "A pi creature lived in a land..."

- GPT-2: incoherent, random continuation.
- GPT-3: "A pi creature lived in a land of math and computation," showing inference and contextual imagination.

3. How Data Flows Through a Transformer (High-Level Preview)

1. Tokenization

Input text is broken into **tokens**: words, sub-words, punctuation, or for multimodal models, patches of images or chunks of audio.

2. Embeddings

Each token is mapped to a high-dimensional **vector** via an **embedding matrix**.

- These vectors act as coordinates in a semantic space.
- Words with similar meanings → vectors close together.

3. Attention Block

Vectors interact. Each token "looks" at others to decide which are important for its meaning.

- Example: The word *model* means different things in *fashion model* vs. *ML model*.
- Attention allows "model" to align with the correct context.

4. Feed-Forward Block (Multi-Layer Perceptron)

Each vector independently goes through the same transformation.

- Intuition: ask each vector a long list of questions ("is it plural?", "is it a person?", "is it technical?"), then update based on the answers.

5. Stacking Layers

Transformers alternate **attention** and **feed-forward** blocks, many times, gradually refining the context-rich representation of tokens.

6. Final Output

A final vector representation is mapped to a distribution over the vocabulary, giving probabilities for the next token.

4. Deep Learning Context

Why transformers work so well connects back to general deep learning principles.

- **Machine Learning:** Instead of writing explicit rules, we learn patterns from data.
- **Linear regression example:** house price vs. square footage. Model has two parameters (slope, intercept).
- **Deep Learning models:** same idea but with billions of parameters.

Parameters = Weights

- **Weights** are the “knobs” the model tunes during training.
- All operations boil down to:
 - Weighted sums (dot products, matrix multiplications).
 - Non-linear activations (ReLU, GELU, etc.).

GPT-3:

- ~175 billion weights.
 - Organized into ~28,000 matrices, across ~8 categories.
-

5. Token Embeddings

First major component: the **Embedding Matrix (WE)**.

- Vocabulary size: ~50,257 tokens.
- Embedding dimension: 12,288.
- Parameters: ~617 million.

What embeddings do:

- Represent tokens as high-dimensional vectors.
- These vectors form a **semantic space** where directions encode meaning.

Classic Examples

- **King – Man + Woman ≈ Queen.**
- **Italy – Germany + Hitler ≈ Mussolini** (capturing nationality + leader relationships).
- **Germany – Japan + Sushi ≈ Bratwurst** (national cuisine analogy).
- **Cat ≈ Beast, Monster** (semantic neighbors).

Plural Direction Example

- Vector for (Cats – Cat) $\approx$ plurality direction.
 - Taking dot products: plural nouns align more strongly with this vector.
 - Numbers “one, two, three...” show increasing alignment with the plurality direction — the model seems to learn a “scale of plurality.”
-

6. Dot Products as Similarity

The **dot product** of two vectors measures alignment:

- Positive $\rightarrow$ vectors point in similar directions.
- Zero $\rightarrow$ perpendicular (unrelated).
- Negative $\rightarrow$ opposite directions.

This intuition is the core of the **attention mechanism**, where relevance between tokens is measured by dot products.

7. Context Size

Transformers operate on a fixed-length **context window**.

- GPT-3: 2,048 tokens.
 - This limits how much conversation/history the model can consider at once.
 - Explains why earlier ChatGPT versions often “lost the thread” in long conversations.
-

8. The Output: Unembedding

At the end, the model needs to produce a distribution over all possible next tokens.

- **Unembedding Matrix (WU):**
 - Shape: vocab size $\times$ embedding dimension.
 - ~ 617 million parameters.
 - Essentially the transpose of the embedding matrix.

Together:

- Embedding + Unembedding matrices = ~ 1.2 B parameters.

- Still only a fraction of GPT-3's 175B total.
-

9. Softmax and Logits

The raw output (from unembedding) = **logits** (unnormalized scores).

- **Softmax** converts logits → probabilities:
 1. Compute $e^{(\text{logit})}$ for each token.
 2. Divide each by the sum of all exponentials.

Result:

- All values $\in (0, 1)$.
- Sum to 1.
- Larger logits dominate, smaller ones shrink toward 0.

Example:

If context is *"least favorite teacher ... Harry Potter ... Professor"*,

- A well-trained model assigns highest probability to **Snape**.

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

where:

- $z = (z_1, z_2, \dots, z_n)$ is the input vector (e.g. the raw attention scores from dot products).
 - $\text{softmax}(z_i)$ is the normalized value for the i -th entry.
 - The denominator is just the sum of exponentials of all entries, making sure the outputs are between 0 and 1 and add up to 1.
-

10. Temperature and Creativity

Softmax can include a **temperature parameter (T)**:

- **T = 0** → deterministic. Always picks max probability. Results are safe, repetitive, boring.
 - Example: "Once upon a time there was A..." → dull Goldilocks-like story.

- $0 < T < 1 \rightarrow$ sharper distribution. Conservative, sticks to safe choices.
- $T > 1 \rightarrow$ distribution flattens. More diversity, more creativity, but also more nonsense.
 - Example: at high T , GPT invents a “young web artist from South Korea” story, but it quickly degenerates.

(OpenAI API caps $T \leq 2$ to avoid complete nonsense).

11. Training Efficiency Note

During training, each token’s vector predicts the **next token after it**, not just the final one.

- This makes training far more efficient.
 - At inference time, we usually only care about the final next-token prediction.
 - Instead of predicting just one token per passage, you predict *all next tokens*, multiplying the amount of data the model learns from.
-

12. Key Takeaways

- Transformers are essentially **giant stacks of matrix multiplications**.
 - To understand them, master:
 - **Embeddings** $\rightarrow$ turning tokens into vectors.
 - **Dot products** $\rightarrow$ measuring similarity.
 - **Softmax** $\rightarrow$ turning scores into probabilities.
 - With these, the core mechanism — **attention** — becomes intuitive.
-

13. Memorable Illustrations

- GPT-2 nonsense vs GPT-3 coherence (“pi creature”).
 - Semantic vector math: *King* – *Man* + *Woman* $\approx$ *Queen*.
 - Political/historical analogy: *Italy* – *Germany* + *Hitler* $\approx$ *Mussolini*.
 - Cultural analogy: *Germany* – *Japan* + *Sushi* $\approx$ *Bratwurst*.
 - Plurality vector: (Cats – Cat) aligns with plural nouns and numbers.
 - Temperature-controlled story generation: Goldilocks vs. creative nonsense.
-

2

1. Big Picture

Transformers are the backbone of modern large language models (LLMs).

- Their job: **take in a sequence of tokens (words/pieces of words)** and predict the next token.
- They do this by **progressively refining word embeddings** — high-dimensional vectors that encode meaning — so that each word becomes contextualized by its surroundings.

Think of an embedding like a coordinate in a huge semantic space. Directions in this space correspond to properties:

- Masculine → Feminine nouns
- Eiffel → Paris/France/steel associations
- Mole → Animal, chemistry, or skin growth depending on context

Transformers let embeddings evolve from *raw lookup meanings* into *contextualized meanings* by passing them through multiple **attention blocks**.

2. Why Attention?

Imagine these examples:

- **American shrew mole** → “mole” = animal
- **One mole of carbon dioxide** → “mole” = chemistry unit
- **Take a biopsy of the mole** → “mole” = skin spot

A simple lookup embedding for “mole” is ambiguous. Attention allows context (surrounding words) to *push* the embedding of “mole” toward the right interpretation.

Another case:

- “Tower” alone → generic “tall building” direction

- “Eiffel tower” → embedding shifts toward Paris/steel
- “Miniature tower” → embedding shifts away from “large/tall” direction

Attention = passing information between embeddings to refine meaning based on context.

3. Ingredients of Self-Attention

Attention operates with three sets of learned matrices:

1. **Query matrix (W_q)**
 - Multiplies the embedding → gives a **query vector (Q)**.
 - Think of it as the “question” a word is asking: “*What am I looking for in other tokens?*”
 - Example: *Creature*’s query could mean: “*Do I have adjectives describing me?*”
 2. **Key matrix (W_k)**
 - Multiplies the embedding → gives a **key vector (K)**.
 - Think of it as the “answer” a word can give if another token queries it.
 - Example: *Fluffy* or *Blue*’s keys align with the query from *Creature*.
 3. **Value matrix (W_v)**
 - Multiplies the embedding → gives a **value vector (V)**.
 - If a word is deemed relevant, *its value is what gets passed along*.
 - Example: *Fluffy* produces a value that, when added to *Creature*, shifts it toward “fluffy creature.” **ADD THE VECTOR DIRECTION INTUITION**
-

4. Computing Relevance (Queries × Keys)

1. Compute dot products between all **queries** and all **keys**.
 - Large positive dot product → strong alignment → token should attend.
 - Example: $q(\text{creature}) \cdot k(\text{fluffy}) = \text{big number}$.
 - $q(\text{creature}) \cdot k(\text{the}) = \text{small number}$.
2. Collect results into a **grid** (attention scores).
3. Apply **softmax** column-wise:
 - Converts scores into weights between 0 and 1.
 - Each column sums to 1.
 - Now we have an **attention pattern** (like a heatmap showing who attends to whom).
4. While the mechanics of queries, keys, and values are mathematically clear, the meaning of what each attention head learns is much more ambiguous. Each head has its own parameter matrices that define a consistent way of producing

queries and keys, but the actual vectors depend on the token embeddings and thus vary with context. After training, some heads appear to specialize in interpretable patterns (like subject-verb agreement, matching brackets, or disambiguating word senses), but many others mix multiple behaviors or remain opaque. In short, we know precisely how attention operates computationally, but the functional role of individual heads is often emergent and only partially understood.

5. Masking

Problem: During training, we predict the next token at every position.

- If later tokens influence earlier ones, the model could “cheat.”

Solution: **Masking**.

- Set scores for “future tokens” $\rightarrow -\infty$ before softmax.
 - After softmax $\rightarrow$ those weights become 0.
 - Ensures no information flows backward in time.
-

6. Updating Embeddings

1. For each column (token), take a weighted sum of **value vectors** (using attention weights).
 - Example: For *Creature*, attention weights on *Fluffy* and *Blue* are high $\rightarrow$ their values dominate the sum.
 2. Add this weighted sum (Δe) to the original embedding.
 - *Creature* embedding shifts to encode “fluffy blue creature.”
 3. Do this for every token simultaneously.
-

7. Single Head vs Multi-Head Attention

- **Single head:** one set of $W_q, W_k, W_v \rightarrow$ captures one kind of relationship (e.g., adjectives $\rightarrow$ nouns).
- **Multi-head:** many sets (GPT-3 has 96 heads per block).
 - Each head learns *different contextual patterns* (syntax, coreference, world knowledge...).

- Outputs from all heads are combined.

This allows the model to encode **many types of context** in parallel.

8. Parameter Counts (GPT-3 scale)

- Embedding dimension: 12,288
 - Query/Key dimension: 128
 - Each head $\approx$ 6.3M parameters
 - 96 heads/block $\rightarrow \approx$ 600M parameters/block
 - 96 layers $\rightarrow \approx$ 58B parameters for attention alone
 - **Total GPT-3 params: 175B** (attention is $\sim \frac{1}{3}$ of the model).
-

9. Variations

- **Cross-attention:** queries from one modality (e.g., English text), keys/values from another (e.g., French text). Used in translation.
 - **Value factoring:** Wv often factored into two smaller matrices (down-projection + up-projection) to save parameters. This is often broken down into **OUTPUT** which is the up-projection matrices of all the heads grouped together and the value matrices which are all the down-projections grouped together.
 - This works in a way that this matrix multiplies the embeddings and basically says, if this embedding attends to another, what vector should be added to the attended embedding to encode this change in meaning.
 - The **value matrix** transforms an embedding into a **value vector** that represents “what information this token can contribute if another token attends to it.”
 - This is done after query-key matrix is computed, and the values are multiplied to the softmaxed query-key matrix. This way only some portions of the value matrix are added to the embedding based on how much the other embedding is relevant.
-

10. Why Attention Works

- Flexible: any token can influence any other (long-range dependencies).
- Parallelizable: dot products and softmax are GPU-friendly.
- Scalable: larger context windows $\rightarrow$ richer understanding (though quadratic cost is a bottleneck).

11. Example Walkthrough

Sentence: “A fluffy blue creature roamed the verdant forest.”

1. Embeddings: $e(\text{fluffy})$, $e(\text{blue})$, $e(\text{creature})$, ...
2. Queries: $q(\text{creature})$ asks “who describes me?”
3. Keys: $k(\text{fluffy})$, $k(\text{blue})$ align well with $q(\text{creature})$.
4. Attention weights: creature attends strongly to fluffy, blue.
5. Values: $v(\text{fluffy})$, $v(\text{blue})$ contain the “fluffiness” and “blueness” adjustments.
6. Update: $e(\text{creature}) \leftarrow e(\text{creature}) + \Delta(\text{fluffy}, \text{blue})$.
 - Result: $e(\text{creature})$ now encodes “fluffy blue creature.”

12. Summary

- **Attention = Queries + Keys + Values.**
- Queries ask, keys answer, values deliver.
- Dot products $\rightarrow$ relevance scores $\rightarrow$ softmax $\rightarrow$ attention pattern.
- Weighted values update embeddings, contextualizing meaning.
- Multi-head attention = many such mechanisms in parallel.
- Transformers stack dozens of these blocks, letting embeddings evolve into carriers of global, nuanced context.

A raw word embedding (like “was”) can, after many layers, encode the full narrative context of a novel — enough to predict the murderer in a mystery story.

1. What the embedding matrix is

- It’s a big lookup table: every token in the vocabulary has a learned vector.
- After training, these vectors are not random — they encode relationships that help the model predict the next word.

2. Where “directions” come from

- When we say “*there’s a gender direction*” or “*a plural direction*”, we mean that if you look at the vectors in this embedding space, **certain differences line up consistently**.
 - Example: king – queen and man – woman both point in a similar direction (gender axis).
 - Example: cat – cats and dog – dogs both point in a similar direction (plural axis).

- These directions are *not explicitly built in* — they **emerge naturally** because the training forces embeddings to capture patterns in language.
-

3. Where this happens

- This kind of structure shows up directly in the **embedding matrix** (the initial token embeddings).
 - As the tokens flow through layers of attention and MLPs, their vectors become more **contextualized** — but the idea of “meaningful directions” is clearest in the learned embedding space itself.
-

3

How Do Large Language Models Store Facts?

1. The Puzzle: Where Do Facts Live in LLMs?

When you give a large language model (LLM) the prompt:

“Michael Jordan plays the sport of ...”

it confidently predicts *“basketball.”*

This means that **somewhere inside its billions of parameters**, the model has *stored knowledge about Michael Jordan* and his sport. But where exactly is that information hiding?

Researchers (including from DeepMind) have been trying to answer this, and while a full understanding is still out of reach, we do know:

- Many facts seem to reside in **MLPs (multi-layer perceptrons)** inside transformers.
 - These MLPs make up the **majority of the parameters** in models like GPT-3.
 - Understanding how they work sheds light on how facts can be represented.
-

2. The Role of MLPs

The **MLP block** is where many facts appear to be stored.

Each token's vector is independently passed through the following steps:

1. **Up projection** – multiply by a huge matrix (W_{up}) and add a bias.
 - Think of each row as a question: “*Does this vector represent Michael Jordan?*”
 - Example: A row equal to *Michael + Jordan* lights up if both names are present.
 2. **Nonlinearity (ReLU/GELU)** – turns this into a **yes/no signal** (active or inactive neuron).
 - Acts like an **AND gate**.
 - Example: Positive only if both *Michael* and *Jordan* are present.
 3. **Down projection** – multiply by another matrix (W_{down}).
 - Here, each column represents a *direction to add back to the embedding*.
 - Example: If the “Michael Jordan neuron” activates, the *basketball direction* is added to the vector.
 4. **Residual connection** – the original vector + this new contribution → passed forward.
-

3. A Toy Example: Encoding “Michael Jordan plays basketball”

Let's assume:

- One direction encodes “*Michael.*”
- Another encodes “*Jordan.*”
- Another encodes “*basketball.*”

Now imagine:

- The **up projection** has a row equal to *Michael + Jordan*.
- The dot product with the input vector is large **only if both names are present**.
- After ReLU, this activates a “Michael Jordan neuron.”
- In the **down projection**, this neuron's column = *basketball direction*.
- Thus, whenever the model processes the name *Michael Jordan*, it **adds basketball to the meaning of that token vector**.

That's how, in principle, a fact can live inside an MLP.

4. Why Neurons Don't Correspond to Simple Features

The toy example suggests neat, interpretable neurons. But reality is messier:

- **Superposition hypothesis:**
 - Instead of one neuron = one fact, features are *superimposed* across many neurons.
 - Why? Because in high dimensions, you can cram in exponentially many nearly-independent directions.
- Example: In a 100-dimensional space, you can represent far more than 100 independent-ish features if you allow them to be *almost perpendicular*.
- Result:
 - A fact like "Michael Jordan → basketball" is not a single neuron firing.
 - It's a pattern spread across many neurons.

5. Key Intuition

- **Attention = context flow** (which words talk to each other).
 - **MLPs = fact storage + feature transformation.**
 - Together: Attention says "We're talking about Michael Jordan." MLP says "Michael Jordan → basketball."
-